

DIA 4 — Como Filtrar Exatamente o que Você Procura em um Banco de Dados

Introdução

No capítulo anterior aprendemos a recuperar informações utilizando a instrução `SELECT`. Entretanto, retornar todos os registros de uma tabela nem sempre é o que desejamos.

Imagine um sistema de e-commerce com milhões de produtos cadastrados ou um banco contendo bilhões de registros. Buscar todas as informações para depois filtrá-las na aplicação seria extremamente ineficiente.

É justamente para resolver esse problema que utilizamos a cláusula **WHERE**.

O `WHERE` permite selecionar apenas os registros que atendem a uma determinada condição, reduzindo a quantidade de dados retornados pelo banco de dados e tornando as consultas muito mais rápidas e eficientes.

Neste capítulo aprenderemos como utilizar filtros em consultas SQL e conheceremos um dos operadores mais úteis da linguagem: o **BETWEEN**.

A Cláusula WHERE

A cláusula `WHERE` é utilizada para restringir os registros retornados por uma consulta.

Sua sintaxe básica é bastante simples.

```
SELECT coluna(s)
FROM tabela
WHERE condição;
```

Tudo o que estiver após o `WHERE` será interpretado como uma condição lógica.

Somente as linhas que atenderem a essa condição serão retornadas.

Filtrando por Igualdade

O operador mais simples é o operador de igualdade (`=`).

Podemos, por exemplo, localizar um cliente específico através do seu código.

```
SELECT *  
FROM clientes  
WHERE cod_cliente = 1;
```

O resultado retornará apenas o cliente cujo código seja igual a **1**.

Da mesma forma, podemos consultar outro cliente.

```
SELECT *  
FROM clientes  
WHERE cod_cliente = 2;
```

E se pesquisarmos um código inexistente?

```
SELECT *  
FROM clientes  
WHERE cod_cliente = 99;
```

Nesse caso, nenhuma linha será retornada.

Isso demonstra um comportamento importante do SQL: quando nenhuma informação atende à condição especificada, o banco simplesmente retorna um conjunto vazio de resultados, sem gerar erro.

Comparações Numéricas

Além da igualdade, o SQL oferece diversos operadores de comparação.

Os principais são:

Operador	Significado
=	Igual
<> ou !=	Diferente
>	Maior que
<	Menor que
>=	Maior ou igual
<=	Menor ou igual

Esses operadores podem ser utilizados em qualquer coluna numérica, de data ou até mesmo em alguns tipos de texto.

Por exemplo, podemos localizar produtos com estoque inferior a 10 unidades.

```
SELECT nome_produto,  
       qtde_estoque  
FROM produtos  
WHERE qtde_estoque < 10;
```

Da mesma forma, podemos consultar produtos cujo preço seja maior ou igual a R\$ 15,00.

```
SELECT nome_produto,  
       preco  
FROM produtos  
WHERE preco >= 15.00;
```

Observe que o banco retorna apenas os registros que satisfazem exatamente a condição especificada.

Combinando Condições com AND

Na prática, raramente utilizamos apenas uma condição.

Na maioria das aplicações precisamos combinar vários critérios simultaneamente.

Para isso utilizamos os operadores lógicos.

O primeiro deles é o **AND**.

O **AND** significa que **todas as condições devem ser verdadeiras ao mesmo tempo**.

Por exemplo, suponha que desejamos localizar apenas produtos com preço acima de R\$ 15,00 e, ao mesmo tempo, com estoque inferior a 10 unidades.

```
SELECT nome_produto,  
       preco  
FROM produtos  
WHERE preco >= 15.00  
       AND qtde_estoque < 10;
```

Observe que somente os produtos que atenderem às duas regras serão retornados.

Caso apenas uma delas seja verdadeira, o registro será descartado.

Essa é uma característica muito importante do operador **AND**.

Filtrando Registros Relacionados

Também podemos utilizar filtros em tabelas que representam relacionamentos entre entidades.

Considere a tabela `pedidos`.

Se quisermos localizar todos os pedidos realizados pelo cliente de código **1**, basta utilizar:

```
SELECT cod_produto,  
       qtde  
FROM pedidos  
WHERE cod_cliente = 1;
```

Essa consulta retorna apenas os produtos comprados por aquele cliente específico.

Mais adiante veremos como combinar essa consulta com **JOINS**, permitindo exibir o nome do cliente e o nome do produto em vez de apenas seus códigos.

O Operador BETWEEN

Em muitas situações precisamos verificar se um valor está dentro de um determinado intervalo.

Embora possamos escrever duas comparações utilizando `>=` e `<=`, existe uma forma muito mais elegante de fazer isso: utilizando o operador **BETWEEN**.

Sua sintaxe é bastante simples.

```
SELECT coluna(s)  
FROM tabela  
WHERE coluna BETWEEN valor_inicial AND valor_final;
```

O `BETWEEN` verifica se o valor informado está **dentro do intervalo**, incluindo os dois extremos.

Por exemplo, podemos localizar produtos cujo preço esteja entre R\$ 10,00 e R\$ 30,00.

```
SELECT nome_produto,  
       preco  
FROM produtos  
WHERE preco BETWEEN 10 AND 30;
```

Essa consulta é equivalente a escrever:

```
SELECT nome_produto,  
       preco  
FROM produtos  
WHERE preco >= 10  
       AND preco <= 30;
```

Entretanto, o `BETWEEN` torna a consulta mais legível e facilita a compreensão da intenção do desenvolvedor.

BETWEEN com Datas

O operador `BETWEEN` também é amplamente utilizado para trabalhar com datas.

Imagine uma tabela de pedidos contendo uma coluna chamada `data_pedido`.

Podemos recuperar todos os pedidos realizados durante um determinado período.

```
SELECT *  
FROM pedidos  
WHERE data_pedido  
      BETWEEN '2026-01-01'  
            AND '2026-01-31';
```

Esse tipo de consulta é extremamente comum em relatórios financeiros, faturamento, auditorias e dashboards.

Quando Utilizar BETWEEN?

O `BETWEEN` é recomendado sempre que desejarmos trabalhar com intervalos de valores, como:

- Faixas de preço;
- Quantidade em estoque;
- Idades;
- Datas;
- Salários;
- Notas;
- Códigos sequenciais.

Ele melhora a legibilidade da consulta e deixa claro que estamos pesquisando um intervalo.

Conclusão

Neste capítulo aprendemos como restringir os resultados de uma consulta utilizando a cláusula `WHERE`.

Conhecemos os principais operadores de comparação, entendemos como combinar condições utilizando o operador lógico `AND` e aprendemos a utilizar o operador `BETWEEN` para pesquisar intervalos de valores.

Esses recursos representam a base para praticamente todas as consultas SQL utilizadas em aplicações reais.

No próximo capítulo continuaremos explorando filtros e conheceremos operadores extremamente importantes, como **OR**, **NOT**, **IN**, **LIKE** e **IS NULL**, que permitirão construir consultas cada vez mais precisas e poderosas.